# Home Chef Cloud — Chef API Documentation

> **Purpose:** This document defines all required REST API endpoints and JSON data structures for the **Chef module**, derived from the Flutter provider layer.
> Backend team should implement these endpoints exactly as described.

---

## Base URL

```
https://api.homechefcloud.com/v1
```

All requests must include:

```
Content-Type: application/json
Authorization: Bearer <token>
```

---

## Table of Contents



---

## 1. Chef — Orders

> **Provider:** `ChefOrdersProvider`

### Endpoints

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/chef/orders` | Get all current orders for this chef |

| `PATCH` | `/chef/orders/{orderId}/accept` | Accept a pending order |
| `PATCH` | `/chef/orders/{orderId}/cancel` | Cancel a pending order |
| `PATCH` | `/chef/orders/{orderId}/status` | Move order to next status |
| `PATCH` | `/chef/orders/{orderId}/delay` | Add prep time delay to an order |
| `POST` | `/chef/orders/{orderId}/verify-handover` | Verify OTP before handing over to rider |

---

### Order Status Lifecycle

```
pending → incoming → preparing → ready → onTheWay → delivered
                                                  ↘ cancelled
```

---

### `GET /chef/orders`

**Response (200 OK):**
```json
{
 "success": true,
 "orders": [
   {
     "id": "4021",
     "status": "preparing",
     "remainingMinutes": 8,
     "placedTime": "2026-04-27T09:00:00Z",
     "customerName": "Sabbir Ahmed",
     "orderValue": 850.0,
     "preferences": ["Less Oil"],
     "riderStatus": "Searching for rider...",
     "items": [
       {
         "name": "Kacchi Biryani (Full)",
         "quantity": 1,
         "dietaryTags": ["Best Seller"]
       },
       {
```

```
          "name": "Borhani",
          "quantity": 2,
          "dietaryTags": []
        }
      ],
      "riderName": null,
      "riderVehicle": null,
      "riderPhoto": null,
      "riderDistance": null,
      "riderEta": null,
      "riderPlateNumber": null,
      "riderLat": null,
      "riderLng": null,
      "handoverOtp": null,
      "estimatedDeliveryTime": null,
      "actualDeliveryTime": null
    }
  ]
}
```

---

### `PATCH /chef/orders/{orderId}/accept`

**Response (200 OK):**
```json
{
  "success": true,
  "orderId": "4035",
  "newStatus": "incoming"
}
```

---

### `PATCH /chef/orders/{orderId}/cancel`

**Response (200 OK):**
```json
{
  "success": true,
  "orderId": "4035",
  "newStatus": "cancelled"
}
```

```

---

### `PATCH /chef/orders/{orderId}/status`

Advance order to the next logical status.

**Response (200 OK):**
```json
{
 "success": true,
 "orderId": "4021",
 "previousStatus": "preparing",
 "newStatus": "ready"
}
```

---

### `PATCH /chef/orders/{orderId}/delay`

**Request Body:**
```json
{
 "additionalMinutes": 5
}
```

**Response (200 OK):**
```json
{
 "success": true,
 "orderId": "4021",
 "newRemainingMinutes": 13
}
```

---

### `POST /chef/orders/{orderId}/verify-handover`

**Request Body:**
```json
{

```
 "otp": "4021"
}
```

**Response (200 OK):**
```json
{
 "success": true,
 "orderId": "4021",
 "newStatus": "onTheWay"
}
```

**Response (400 Bad Request):**
```json
{
 "success": false,
 "message": "Invalid OTP"
}
```

---

### Data Structure — `Order`

| Field | Type | Nullable | Description |
|---|---|---|---|
| `id` | `string` | No | Unique order ID |
| `status` | `enum` | No | `pending \| incoming \| preparing \| ready \| onTheWay \| delivered \| cancelled` |
| `remainingMinutes` | `int` | No | Remaining prep time in minutes |
| `placedTime` | `ISO8601 string` | No | When the order was placed |
| `customerName` | `string` | No | Customer's name |
| `orderValue` | `float` | Yes | Total value of the order (BDT) |
| `preferences` | `string[]` | No | Customer notes (e.g. "Less Oil") |
| `riderStatus` | `string` | No | Human-readable rider status text |
| `items` | `OrderItem[]` | No | Array of ordered items |
| `riderName` | `string` | Yes | Assigned rider's name |
| `riderVehicle` | `string` | Yes | `"Bike"` or `"Cycle"` |
| `riderPhoto` | `string (URL)` | Yes | Rider profile photo URL |
| `riderDistance` | `float` | Yes | Distance in meters |
| `riderEta` | `int` | Yes | ETA in minutes |
| `riderPlateNumber` | `string` | Yes | Vehicle plate number |
| `riderLat` | `float` | Yes | Rider current latitude |

| `riderLng` | `float` | Yes | Rider current longitude |
| `handoverOtp` | `string` | Yes | OTP for rider handover verification |
| `estimatedDeliveryTime` | `ISO8601 string` | Yes | Expected delivery time |
| `actualDeliveryTime` | `ISO8601 string` | Yes | Actual delivery time |

### Data Structure — `OrderItem`

| Field | Type | Nullable | Description |
|---|---|---|---|
| `name` | `string` | No | Item name |
| `quantity` | `int` | No | Quantity ordered |
| `dietaryTags` | `string[]` | No | e.g. `["Veg", "Spicy", "Best Seller"]` |

---

## 2. Chef — Wallet

> **Provider:** `ChefWalletProvider`

### Endpoints

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/chef/wallet` | Get wallet balance summary |
| `GET` | `/chef/wallet/transactions` | Get transaction history (with date filter) |
| `POST` | `/chef/wallet/withdraw` | Request a withdrawal |

---

### `GET /chef/wallet`

**Response (200 OK):**
```json
{
 "success": true,
 "availableBalance": 45200.0,
 "pendingBalance": 12500.0,
 "currency": "BDT",
 "todayEarnings": 620.0,
 "todayOrderCount": 2,
```

```
 "weeklyGrowthPercent": 15.4,
 "weeklySparkline": [320, 450, 280, 780, 620, 850, 1250]
}
```

---

### `GET /chef/wallet/transactions?filter=week`

**Query Params:**

| Param | Values | Default | Description |
|---|---|---|---|
| `filter` | `today \| week \| month` | `week` | Date range filter |

**Response (200 OK):**
```json
{
 "success": true,
 "filter": "week",
 "totalEarnings": 2900.0,
 "orderCount": 5,
 "avgOrderValue": 580.0,
 "transactions": [
   {
     "id": "TXN-001",
     "orderId": "4019",
     "date": "2026-04-27T07:00:00Z",
     "amount": 450.0,
     "type": "processing",
     "description": "Order Sale",
     "customerName": "Ashikur Rahman",
     "orderItems": ["Protein Platter x1"]
   },
   {
     "id": "TXN-003",
     "orderId": null,
     "date": "2026-04-26T09:00:00Z",
     "amount": 5000.0,
     "type": "debit",
     "description": "Withdrawal to bKash",
     "customerName": null,
     "orderItems": null
   }
 ]
```

}
```

---

### `POST /chef/wallet/withdraw`

**Request Body:**
```json
{
 "amount": 5000.0,
 "method": "bkash",
 "accountDetails": "01712345678"
}
```

**Response (200 OK):**
```json
{
 "success": true,
 "transactionId": "TXN-W1745741600000",
 "amount": 5000.0,
 "method": "bkash",
 "newAvailableBalance": 40200.0,
 "status": "processing"
}
```

**Response (400 Bad Request):**
```json
{
 "success": false,
 "message": "Insufficient balance or amount below minimum (500 BDT)"
}
```

---

### Data Structure — `WalletTransaction`

| Field | Type | Nullable | Description |
|---|---|---|---|
| `id` | `string` | No | Unique transaction ID |
| `orderId` | `string` | Yes | Related order ID (null for withdrawals/fees) |
```

| `date` | `ISO8601 string` | No | Transaction date/time |
| `amount` | `float` | No | Transaction amount (BDT) |
| `type` | `enum` | No | `credit \| debit \| processing` |
| `description` | `string` | No | e.g. `"Order Sale"`, `"Withdrawal to bKash"`, `"Platform Fee"` |
| `customerName` | `string` | Yes | Customer name for order-type transactions |
| `orderItems` | `string[]` | Yes | List of item strings, e.g. `["Biryani x1"]` |

### Enum — `WithdrawalMethod`

| Value | Label |
|---|---|
| `bkash` | bKash |
| `nagad` | Nagad |
| `bank` | Bank Card |

---

## 3. Chef — Recipes

> **Provider:** `ChefRecipesProvider`

### Endpoints

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/chef/recipes` | Get all recipes for the chef |
| `POST` | `/chef/recipes` | Add a new recipe (starts as `pending`) |
| `PUT` | `/chef/recipes/{recipeId}` | Update an existing recipe |
| `DELETE` | `/chef/recipes/{recipeId}` | Delete a recipe |

---

### `GET /chef/recipes`

**Response (200 OK):**
```json
{
 "success": true,
 "recipes": [
   {
     "id": "r1",
     "name": "Kacchi Biryani (Full)",
```

```
     "description": "Authentic mutton kacchi biryani cooked with
aromatic basmati rice.",
     "price": 450.0,
     "imageUrl": "https://images.unsplash.com/...",
     "category": "Main Course",
     "preparationTime": 45,
     "dietaryTags": ["Best Seller", "Meat"],
     "status": "approved",
     "createdAt": "2026-04-17T00:00:00Z"
   }
 ]
}
```

---

### `POST /chef/recipes`

**Request Body:**
```json
{
 "name": "Grilled Salmon",
 "description": "Premium salmon fillet with buttered asparagus.",
 "price": 1250.0,
 "imageUrl": "https://...",
 "category": "Premium",
 "preparationTime": 25,
 "dietaryTags": ["Seafood", "Keto"]
}
```

**Response (201 Created):**
```json
{
 "success": true,
 "recipe": {
   "id": "r8",
   "name": "Grilled Salmon",
   "description": "Premium salmon fillet with buttered asparagus.",
   "price": 1250.0,
   "imageUrl": "https://...",
   "category": "Premium",
   "preparationTime": 25,
   "dietaryTags": ["Seafood", "Keto"],
   "status": "pending",
```

```
   "createdAt": "2026-04-27T09:00:00Z"
 }
}
```

---

### `PUT /chef/recipes/{recipeId}`

**Request Body:** *(all fields optional)*
```json
{
 "price": 1300.0,
 "dietaryTags": ["Seafood", "Keto", "Gluten Free"]
}
```

**Response (200 OK):**
```json
{
 "success": true,
 "recipe": { "...updated recipe object..." }
}
```

---

### `DELETE /chef/recipes/{recipeId}`

**Response (200 OK):**
```json
{
 "success": true,
 "message": "Recipe deleted successfully"
}
```

---

### Data Structure — `Recipe`

| Field | Type | Nullable | Description |
|---|---|---|---|
| `id` | `string` | No | Unique recipe ID |
| `name` | `string` | No | Recipe name |

| `description` | `string` | No | Short description |
| `price` | `float` | No | Price in BDT |
| `imageUrl` | `string (URL)` | No | Recipe image |
| `category` | `string` | No | e.g. `"Main Course"`, `"Drinks"`, `"Fast Food"`, `"Healthy"`, `"Premium"` |
| `preparationTime` | `int` | No | Time in minutes |
| `dietaryTags` | `string[]` | No | e.g. `["Veg", "Spicy", "Best Seller", "Keto"]` |
| `status` | `enum` | No | `approved \| pending \| rejected` |
| `createdAt` | `ISO8601 string` | No | Creation timestamp |

---

## 4. Chef — Reviews

> **Provider:** `ChefReviewsProvider`

### Endpoints

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/chef/reviews` | Get all customer reviews for this chef |
| `POST` | `/chef/reviews/{reviewId}/reply` | Chef replies to a customer review |
| `GET` | `/chef/reviews/unrated-riders` | Get delivered orders needing rider ratings |
| `POST` | `/chef/reviews/rate-rider` | Submit a rating for a rider |

---

### `GET /chef/reviews`

**Response (200 OK):**
```json
{
 "success": true,
 "averageRating": 4.5,
 "reviews": [
   {
     "id": "REV-001",
     "targetId": "REC-1",
     "targetName": "Spicy Fish Curry",
     "customerName": "Ashikur Rahman",
     "rating": 5.0,
     "comment": "Absolutely delicious!",
```

```
      "date": "2026-04-26T09:00:00Z",
      "type": "customerToChef",
      "chefReply": null,
      "replyDate": null
    }
  ]
}
```

---

### `POST /chef/reviews/{reviewId}/reply`

**Request Body:**
```json
{
  "reply": "Thank you for your kind words!"
}
```

**Response (200 OK):**
```json
{
  "success": true,
  "reviewId": "REV-001",
  "reply": "Thank you for your kind words!",
  "replyDate": "2026-04-27T10:00:00Z"
}
```

---

### `GET /chef/reviews/unrated-riders`

**Response (200 OK):**
```json
{
  "success": true,
  "unratedOrders": [
    {
      "orderId": "4025",
      "customerName": "Anika Tabassum",
      "riderName": "Kamal Hossain",
      "deliveredAt": "2026-04-27T08:00:00Z",
      "items": [
```

```
       { "name": "Spicy Fish Curry", "quantity": 1, "dietaryTags": []
}
     ]
   }
 ]
}
```

---

### `POST /chef/reviews/rate-rider`

**Request Body:**
```json
{
 "orderId": "4025",
 "riderName": "Kamal Hossain",
 "rating": 4.5,
 "comment": "Very punctual and professional.",
 "metrics": {
   "Punctuality": 5.0,
   "Behavior": 4.5,
   "Packaging Care": 4.0
 }
}
```

**Response (201 Created):**
```json
{
 "success": true,
 "reviewId": "REV-R-4025",
 "message": "Rider rated successfully"
}
```

---

### Data Structure — `Review`

| Field | Type | Nullable | Description |
|---|---|---|---|
| `id` | `string` | No | Unique review ID |
| `targetId` | `string` | No | Recipe ID or Rider ID |
| `targetName` | `string` | No | Recipe name or Rider name |

| `customerName` | `string` | Yes | Customer name (for `customerToChef` type) |
| `riderImage` | `string (URL)` | Yes | Rider photo (for `chefToRider` type) |
| `rating` | `float` | No | Rating value (1.0 - 5.0) |
| `comment` | `string` | No | Review text |
| `date` | `ISO8601 string` | No | Review date |
| `type` | `enum` | No | `customerToChef \| chefToRider` |
| `chefReply` | `string` | Yes | Chef's reply to customer review |
| `replyDate` | `ISO8601 string` | Yes | When the reply was submitted |
| `metrics` | `object` | Yes | Rider rating metrics — `{ "Punctuality": 5.0, "Behavior": 4.5 }` |

---

## 5. Chef — Live Stream

> **Provider:** `ChefLiveStreamProvider`

### Endpoints

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/chef/livestream/start` | Start a live cooking stream |
| `POST` | `/chef/livestream/stop` | Stop the current live stream |
| `PATCH` | `/chef/livestream/step` | Update the current cooking step |
| `GET` | `/chef/livestream/status` | Get current stream status |

---

### `POST /chef/livestream/start`

**Request Body:**
```json
{
 "orderId": "4021"
}
```

**Response (200 OK):**
```json
{
 "success": true,
```

```
 "streamId": "stream-abc123",
 "orderId": "4021",
 "startTime": "2026-04-27T09:00:00Z",
 "viewerCount": 1,
 "currentStep": "Preparing Ingredients",
 "cookingSteps": [
   "Preparing Ingredients",
   "Heating Oil & Spices",
   "Marinating Meat",
   "Adding Rice & Saffron",
   "Layering & Dum Cooking",
   "Final Garnishing"
 ]
}
```

---

### `POST /chef/livestream/stop`

**Response (200 OK):**
```json
{
 "success": true,
 "message": "Stream ended"
}
```

---

### `PATCH /chef/livestream/step`

**Request Body:**
```json
{
 "step": "Heating Oil & Spices"
}
```

**Response (200 OK):**
```json
{
 "success": true,
 "currentStep": "Heating Oil & Spices"
}
```

```

---

### `GET /chef/livestream/status`

**Response (200 OK):**
```json
{
 "success": true,
 "isStreaming": true,
 "orderId": "4021",
 "viewerCount": 12,
 "startTime": "2026-04-27T09:00:00Z",
 "currentStep": "Marinating Meat"
}
```

---

### Data Structure — `LiveStreamSession`

| Field | Type | Nullable | Description |
|---|---|---|---|
| `streamId` | `string` | No | Unique stream session ID |
| `orderId` | `string` | No | Linked order ID |
| `isStreaming` | `bool` | No | Whether stream is active |
| `viewerCount` | `int` | No | Current viewer count (min: 1) |
| `startTime` | `ISO8601 string` | Yes | When stream started |
| `currentStep` | `string` | No | Current cooking step label |
| `cookingSteps` | `string[]` | No | All available cooking step labels |

---

## 6. Chef — Profile

> **Provider:** `ChefProfileProvider`

### Endpoints

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/chef/profile` | Get chef profile information |
| `PATCH` | `/chef/profile` | Update name or email |
```

| `POST` | `/chef/profile/avatar` | Upload a new profile photo |

---

### `GET /chef/profile`

**Response (200 OK):**
```json
{
 "success": true,
 "profile": {
   "id": "chef-001",
   "name": "Chef Farhana",
   "email": "farhana.khan@homechef.com",
   "phone": "01712345678",
   "profileImageUrl": "https://images.unsplash.com/..."
 }
}
```

---

### `PATCH /chef/profile`

**Request Body:** *(all fields optional)*
```json
{
 "name": "Chef Farhana Khan",
 "email": "farhana.new@homechef.com"
}
```

**Response (200 OK):**
```json
{
 "success": true,
 "profile": {
   "id": "chef-001",
   "name": "Chef Farhana Khan",
   "email": "farhana.new@homechef.com",
   "profileImageUrl": "https://..."
 }
}
```

---

### `POST /chef/profile/avatar`

**Request:** `multipart/form-data`

| Field | Type | Description |
|---|---|---|
| `avatar` | `file` | Image file (JPG / PNG) |

**Response (200 OK):**
```json
{
 "success": true,
 "profileImageUrl":
"https://cdn.homechefcloud.com/avatars/chef-001.jpg"
}
```

---

### Data Structure — `ChefProfile`

| Field | Type | Nullable | Description |
|---|---|---|---|
| `id` | `string` | No | Chef's user ID |
| `name` | `string` | No | Display name |
| `email` | `string` | No | Email address |
| `phone` | `string` | No | Phone number |
| `profileImageUrl` | `string (URL)` | No | Profile photo URL |

---

## Global Error Response Format

All error responses follow this structure:

```json
{
 "success": false,
 "message": "Human-readable error description",
 "code": "ERROR_CODE_SLUG"
}
```

**Common HTTP Status Codes:**

| Code | Meaning |
|---|---|
| `200` | Success |
| `201` | Resource created |
| `400` | Bad request / validation error |
| `401` | Unauthenticated |
| `403` | Forbidden |
| `404` | Resource not found |
| `500` | Internal server error |

---

*Document generated from Flutter provider source code — Home Chef Cloud v1.0.0*
*Author: Raufuzzaman Ashik*